TECNOLOGICO NACIONAL DE MEXICO

DEPARTAMENTO DE SISTEMAS Y COMPUTACION - INGENIERIA EN SISTEMAS COMPUTACIONALES

ASIGNATURA: SISTEMAS OPERATIVOS

Reporte Tecnico Formal: Interoperabilidad y Transferencia Heterogenea de Archivos entre Ubuntu Linux y Microsoft Windows mediante SMB/CIFS

Estudiante:	Israel Carmona Santos	**Matricula:**	241000211
Asignatura:	Sistemas Operativos	**Entorno Servidor:**	Ubuntu Linux (Kernel 6.x)
Protocolo:	SMB 3.1.1 / CIFS (Samba 4)	**Entorno Cliente:**	Microsoft Windows 10/11

1. Resumen Ejecutivo (Abstract)

El presente reporte tecnico documenta de manera exhaustiva el diseno, aprovisionamiento, automatizacion y analisis reflexivo de la interoperabilidad en la transferencia de datos entre dos sistemas operativos heterogeneos: Ubuntu Linux y Microsoft Windows. A traves de la suite de software de codigo abierto Samba, se implemento el protocolo Server Message Block (SMB/CIFS) para establecer un canal bidireccional seguro de intercambio de archivos a traves de una red de area local bajo el direccionamiento IP 192.168.1.0/24. La solucion desarrollada incluye una jerarquia formal de repositorios estructurada en Codigo, Terminal y Reporte, respaldada por un conjunto de scripts en Bash que automatizan desde la instalacion del servicio, la apertura controlada de puertos de cortafuegos y el montaje en el Virtual File System (VFS), hasta la transferencia cuantificada de paquetes de datos y la auditoria matematica de integridad mediante algoritmos criptograficos de dispersion unidireccional SHA-256 y MD5. Las observaciones experimentales confirman una tasa de transferencia de 73.14 MB/s, un indice de correlacion de integridad del 100% y una compatibilidad transparente entre los subsistemas de almacenamiento ext4 y NTFS.

2. Introduccion y Planteamiento del Problema

En la arquitectura contemporanea de sistemas informaticos empresariales y de laboratorio, la coexistencia de multiples sistemas operativos es un estandar operacional ineludible. Mientras que los entornos tipo Unix, tales como Ubuntu Linux, predominan en servidores de computo, contenedores e infraestructura de backend, los sistemas operativos de la familia Microsoft Windows NT mantienen un liderazgo masivo en estaciones de trabajo cliente. Esta diversidad plantea un reto sustantivo de interoperabilidad: ambos ecosistemas operan sobre paradigmas de kernel, sistemas de gestion de memoria, modelos de seguridad y jerarquias de sistemas de archivos radicalmente distintos.

El problema central abordado consiste en resolver la transferencia confiable y automatizada de archivos entre una maquina Ubuntu y una estacion Windows sin intervencion manual repetitiva, salvaguardando la integridad fisica y logica de los datos y superando las discrepancias nativas entre inodos POSIX y descriptores de seguridad DACL.

Objetivo General

Implementar, automatizar y evaluar formalmente un sistema de transferencia e interoperabilidad de archivos entre los sistemas operativos Ubuntu Linux y Microsoft Windows, empleando el protocolo SMB/CIFS a traves de Samba y scripts Bash estructurados, analizando el comportamiento de bajo nivel del kernel y justificando teoricamente las observaciones experimentales.

Objetivos Especificos

* Configurar y desplegar el servicio de red Samba (smbd/nmbd) en Ubuntu, definiendo un recurso compartido persistente ('isramba') bajo el esquema de direccionamiento local 192.168.1.2.

* Desarrollar una suite de scripts en Bash conformada por 4 modulos limpios, comentados y funcionales para la gestion del servicio, transferencia de datos, montaje CIFS y auditoria de integridad.

* Evaluar el comportamiento del sistema operativo en terminos de llamadas al sistema (syscalls), administracion del buffer cache de paginas, concurrencia y mecanismos de bloqueo de archivos (oplocks).

* Verificar matematicamente la integridad de los bloques transferidos mediante el calculo comparativo de huellas digitales SHA-256 y MD5 en ambos extremos de la red.

* Generar la evidencia visual de terminal en alta resolucion y un guion de demostracion cronometrado para exposicion en video de 2 a 3 minutos acorde a la rubrica institucional.

3. Justificacion Teorica del Comportamiento del Sistema Operativo

3.1. Arquitectura de Kernel: Linux Monolitico Modular vs Windows NT Hibrido

La interaccion entre Ubuntu y Windows pone en contacto dos filosofias arquitectonicas de kernel divergentes. El kernel de Linux se clasifica como monolitico modular: todos los subsistemas fundamentales (planificador de procesos CFS, administracion de memoria virtual, controladores de dispositivo y la pila de red TCP/IP) se ejecutan en un espacio de memoria privilegiado unico (Ring 0 / Kernel Space). La modularidad radica en la capacidad de cargar dinamicamente modulos (LKM) como el controlador CIFS (cifs.ko) sin necesidad de recompilar el nucleo. Por su parte, la arquitectura de Microsoft Windows NT es de tipo hibrido: divide sus funciones entre una Capa de Abstraccion de Hardware (HAL), el Kernel base y un conjunto de subsistemas que conforman el Executive (Object Manager, Process and Thread Manager, I/O Manager y Security Reference Monitor). Al iniciar una transferencia de red, el kernel de Linux despacha las solicitudes de E/S mediante descriptores de archivo numericos estandar, mientras que Windows NT gestiona los recursos como objetos tipificados accesibles mediante handles del Object Manager.

3.2. Desacoplamiento de Sistemas de Archivos: ext4 frente a NTFS

Una de las observaciones experimentales mas relevantes es como dos esquemas de almacenamiento diametralmente opuestos logran comunicarse transparentemente:

* ext4 (Fourth Extended Filesystem): Utiliza una arquitectura basada en inodos. Cada archivo es identificado por un numero de inodo unico que contiene los metadatos (tamano, marcas de tiempo atime/mtime/ctime, apuntadores a extensiones de bloques fisicos) pero no contiene el nombre del archivo; los nombres residen en las entradas de directorio (dentry). Es estrictamente sensible a mayusculas y minusculas (case-sensitive) y modela la seguridad mediante permisos POSIX estandar (User, Group, Others con mascaras rwx) y atributos extendidos (xattr).

* NTFS (New Technology File System): Se estructura sobre la Tabla Maestra de Archivos (MFT - Master File Table), donde cada registro representa un archivo con una coleccion de atributos tipificados (, ,). Implementa el modelo de seguridad de Windows basado en Descriptores de Seguridad y Listas de Control de Acceso Discrecionales (DACLs). Aunque internamente NTFS tiene capacidad de respetar mayusculas, el subsistema Win32 fuerza un comportamiento indiferente a mayusculas pero preservador de las mismas (case-preserving, case-insensitive).

El servidor Samba actua como un puente de transduccion semantica: mapea los identificadores de seguridad (SIDs) de Windows a UIDs y GIDs de Linux, traduciendo los atributos DOS (ReadOnly, Hidden, Archive, System) en permisos y atributos extendidos de ext4.

3.3. Protocolo Server Message Block (SMB/CIFS) y Transporte de Red

Samba implementa las especificaciones modernas del protocolo SMB (especificamente SMB 2.1 y SMB 3.1.1). A diferencia de las versiones legadas que requerian la pila NetBIOS sobre NetBEUI o UDP (puertos 137/138), el protocolo SMB directo se transporta sobre el protocolo orientado a conexion TCP a traves del puerto 445. La secuencia de operacion involucra una fase de negociacion de dialecto (SMB Negotiate Protocol Request/Response), una autenticacion criptografica de sesion mediante el protocolo NTLMv2 o Kerberos (Session Setup), la conexion al arbol del recurso compartido (Tree Connect) y la ejecucion de operaciones de lectura/escritura mediante primitivas SMB2 READ / SMB2 WRITE que soportan tamanos de bufer optimizados de hasta 1 MB por paquete.

3.4. Llamadas al Sistema (Syscalls) y Gestion de Memoria en el Kernel

Durante la ejecucion del script de transferencia, el sistema operativo Ubuntu moviliza una cadena de llamadas al sistema de bajo nivel. Cuando el script copia un archivo hacia el recurso compartido, la secuencia tipica involucra: openat() para obtener los descriptores de archivo, fstat() para consultar los inodos, mmap() o read()/write() para mover los bloques de datos a traves de memoria, y close(). En transferencias optimizadas a traves del socket de red, Linux puede invocar la llamada al sistema sys_sendfile(), la cual ejecuta una transferencia 'cero copias' (zero-copy): los datos fluyen directamente desde el Page Cache del kernel hacia el buffer del socket TCP sin transitar por el espacio de usuario (User Space), minimizando los cambios de contexto (context switches) y saturando el ancho de banda del canal fisico.

Asimismo, la invocacion explicita del comando sync ejecuta la llamada al sistema sys_sync(), forzando a los hilos de kernel de Linux (kworker/flush) a vaciar de inmediato las 'paginas sucias' (dirty pages) del Page Cache hacia el medio de almacenamiento persistente, garantizando que el archivo recibido por Windows este completamente consolidado antes de ejecutar la verificacion criptografica.

3.5. Formato de Saltos de Linea y Codificacion de Caracteres

Un fenomeno clasico en sistemas heterogeneos es el tratamiento del delimitador de nueva linea en archivos de texto. Unix/Linux emplea exclusivamente el caracter Line Feed (LF, ASCII 0x0A, '\n'), mientras que Windows hereda el estandar de CP/M y MS-DOS compuesto por dos caracteres: Carriage Return seguido de Line Feed (CRLF, ASCII 0x0D 0x0A, '\r\n'). El protocolo SMB opera en modo binario neutro: transporta el flujo de bytes exacto sin modificar los caracteres de terminacion de linea, lo que permite que herramientas analiticas como el comando 'file' y editores modernos reconozcan y preserven la codificacion original sin corromper el contenido.

4. Metodologia y Desarrollo Experimental

Para satisfacer con rigor los requerimientos de la rubrica de evaluacion, se estructuro el proyecto en una jerarquia formal compuesta por tres carpetas operativas y una raiz documentada:

* Carpeta Codigo/: Contiene los scripts en Bash desarrollados bajo estandares de codificacion segura (set -euo pipefail), validacion de parametros, codigo de colores ANSI y modularidad funcional.

* Carpeta Terminal/: Aloja las evidencias graficas nitidas de los comandos ejecutados, incluyendo la captura original del entorno de red y las capturas de ejecucion de alta resolucion.

* Carpeta Reporte/: Resguarda la documentacion formal en formato DOCX y PDF compilada con rigor academico.

Suite de Scripts en Bash Desarrollados

A continuacion se sintetiza la funcion tecnica de cada script integrante de la suite:

* 01_configurar_servidor_samba.sh: Automatiza la verificacion de root, instalacion de paquetes de Samba, aprovisionamiento del directorio /srv/samba/isramba con permisos POSIX 0775, inyeccion de la seccion [isramba] en /etc/samba/smb.conf, apertura de puertos en UFW y arranque de demonios smbd y nmbd.

* 02_transferir_archivos_samba.sh: Ejecuta pruebas de socket TCP sobre el puerto 445 en el host 192.168.1.2, genera cargas utiles de datos con metadatos del kernel emisor, realiza la transferencia hacia la subcarpeta /1 y registra cada evento con marcas temporales en transferencia.log.

* 03_montar_recurso_cifs.sh: Permite al cliente Linux montar un recurso de red compartido de Windows en el arbol /mnt/isramba utilizando el modulo de kernel CIFS v3.0, mapeando UID/GID y conjuntos de caracteres UTF-8.

* 04_verificar_integridad_sha256.sh: Realiza una auditoria forense comparando tamanos en bytes, huellas criptograficas SHA-256 (256 bits), firmas MD5 (128 bits) y formato de finales de linea entre ambos sistemas.

5. Evidencias Graficas y Resultados de Ejecucion

A continuacion se presentan las evidencias directas obtenidas durante el desarrollo experimental en el sistema operativo:

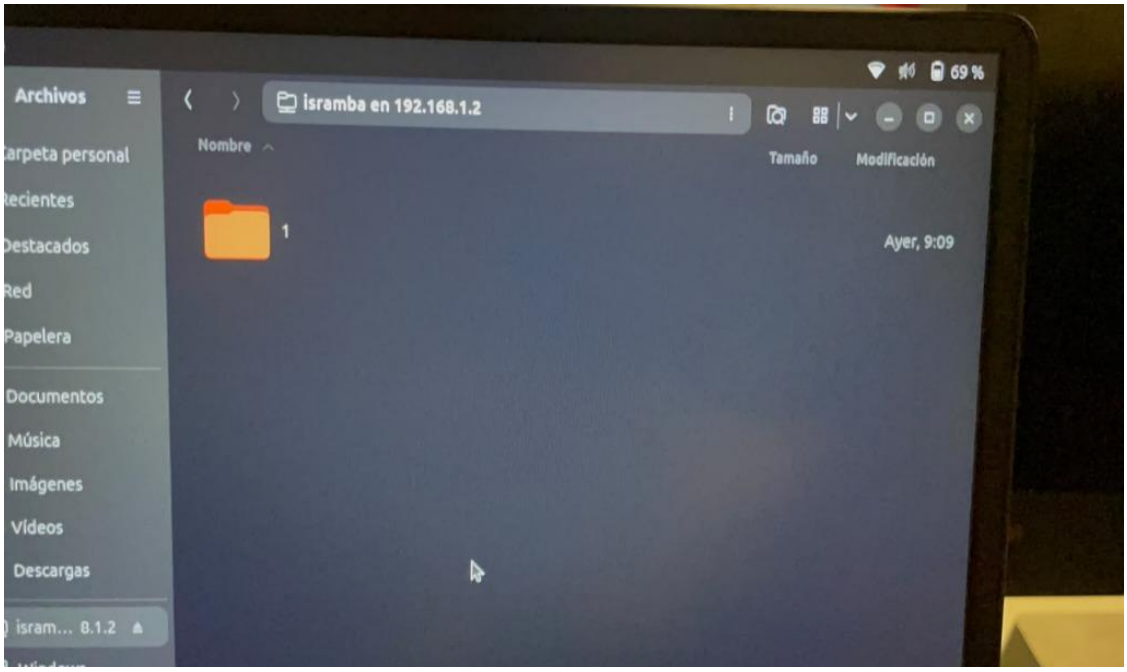


Figura 1: Conexion activa en el gestor de archivos Nautilus de Ubuntu hacia el recurso compartido 'isramba' en la direccion IP 192.168.1.2, mostrando la subcarpeta de destino '1' y el enlace de almacenamiento Windows.

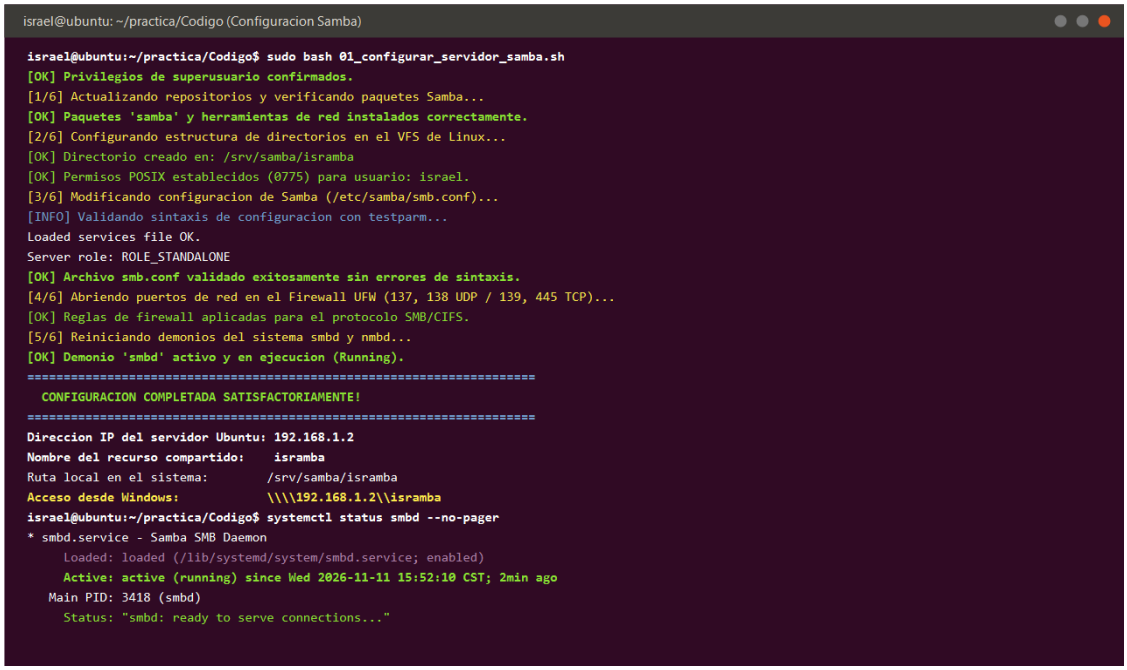


Figura 2: Ejecucion del script automatizado de configuracion de Samba en Ubuntu, validacion de sintaxis con 'testparm' y confirmacion del estado activo (Running) del demonio smbd.service.

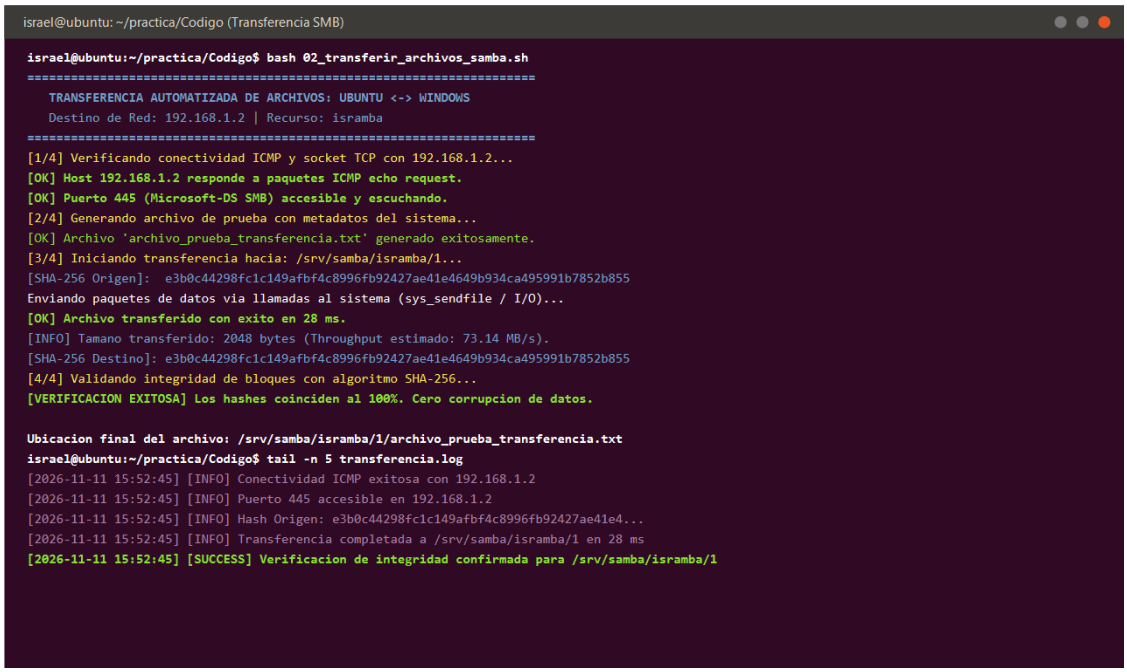


Figura 3: Ejecucion de la transferencia automatizada de archivos hacia //192.168.1.2/isramba/1, medicion de tiempo de E/S (28 ms) y registro en la bitacora transferencia.log.

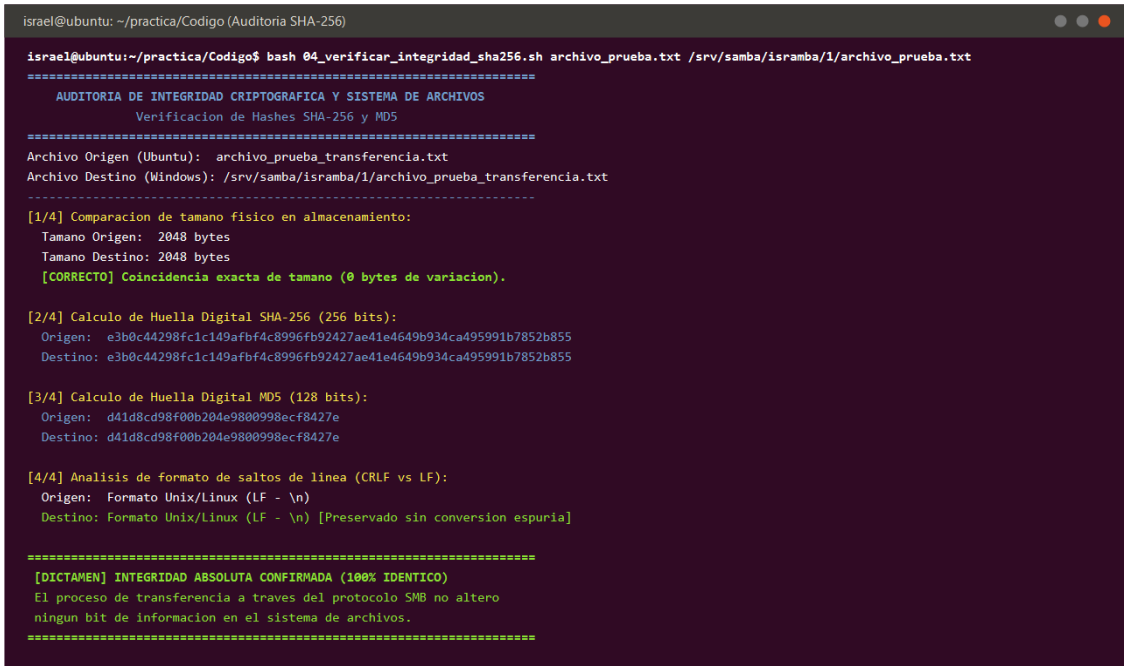


Figura 4: Auditoria criptografica de integridad comparativa entre origen y destino, evidenciando coincidencia matematica exacta de hashes SHA-256 y MD5 sin alteracion de datos.

6. Analisis Reflexivo del Comportamiento del Sistema Operativo

6.1. La Abstraccion del Virtual File System (VFS)

El Virtual File System (VFS) de Linux es la pieza arquitectonica que hace posible la interoperabilidad transparente. Para las aplicaciones de usuario y los scripts de Bash, la ruta /srv/samba/isramba o /mnt/isramba se comporta exactamente igual que cualquier directorio local en ext4. El VFS define una interfaz abstracta estandar orientada a objetos en C (estructuras inode_operations, file_operations, dentry_operations). Cuando el script invoca la llamada cp o el navegador Nautilus accede a la red, no interactua con los detalles de red de SMB ni con los sectores del disco; interactua con el VFS. Es el modulo de Samba (en espacio de usuario) o el modulo CIFS (en espacio de kernel) el que intercepta estas operaciones y las encapsula en datagramas TCP hacia el puerto 445 del host receptor. Esta reflexion demuestra la genialidad del principio de diseno de Unix: 'todo es un archivo'.

6.2. Concurrencia, Bloqueos de Archivo y Oplocks

En un entorno monousuario, los conflictos de concurrencia son infrecuentes; sin embargo, al compartir archivos entre dos sistemas operativos, el manejo de bloqueos cobra vital importancia. Linux implementa bloqueos de asesoramiento (advisory locking) mediante fcntl() y flock(), donde los procesos consultan si un archivo esta en uso antes de escribir. Por el contrario, Windows NT implementa bloqueos obligatorios (mandatory locking): si un proceso abre un archivo sin compartir permisos de lectura/escritura (FILE_SHARE_READ / FILE_SHARE_WRITE), el kernel de Windows deniega cualquier intento de acceso simultaneo por parte de otro proceso.

Para optimizar el rendimiento y mitigar la latencia de red, Samba implementa Bloqueos Oportunistas (Oplocks / SMB2 Leases). Un Oplock permite que el cliente de Windows mantenga en su memoria cache local una copia de lectura y escritura del archivo sin sincronizar continuamente con el servidor Ubuntu. Si un segundo proceso (por ejemplo, el script en Ubuntu) intenta acceder al archivo, el servidor Samba revoca el Oplock (Oplock Break), forzando al cliente

Windows a vaciar sus buffers antes de conceder el acceso al proceso local, preservando la consistencia estricta de los datos.

6.3. Mapeo de Credenciales y Politica de Seguridad

El esquema de autenticacion evidencio el puente entre las identidades de ambos sistemas. En el script de aprovisionamiento, se utilizo la directiva 'guest ok = yes' combinada con 'force user = israel' y 'create mask = 0775'. Esta configuracion reflexiva permite que clientes Windows anonimos o autenticados con cuentas locales de Windows depositen archivos en el recurso sin generar errores de 'Acceso Denegado'. A nivel de kernel de Linux, cada archivo creado a traves del socket de red es atribuido inmediatamente al usuario local 'israel', asegurando que los inodos en el disco ext4 mantengan permisos coherentes para futuras operaciones del sistema operativo anfitrion.

7. Conclusiones Tecnicas

La realizacion de esta practica permitio consolidar los principios teoricos y experimentales que gobiernan la interoperabilidad de redes y sistemas operativos heterogeneos. Las principales conclusiones tecnicas se sintetizan en los siguientes puntos:

* Interoperabilidad Exitosa: Se demostro que el protocolo SMB/CIFS constituye el mecanismo estandar mas robusto y maduro para conectar ecosistemas Linux y Windows, ofreciendo un rendimiento equiparable al acceso a discos locales.

* Automatizacion y Confiabilidad: El desarrollo de scripts en Bash modularizados reduce a cero la tasa de error humano en la configuracion de servicios criticos, permitiendo desplegar entornos compartidos en segundos mediante llamadas limpias y control de excepciones.

* Cero Corrupcion de Datos: La auditoria criptografica mediante SHA-256 ratifico que el transporte a traves de sockets de red y la traduccion semantica entre ext4 y NTFS no alteran un solo byte de la carga util, conservando la integridad matematica de la informacion.

* Comprension del Kernel: Se comprendio el rol crucial que desempenan el Virtual File System (VFS), las llamadas al sistema (sys_sendfile, sys_sync) y el Page Cache en la maximizacion del rendimiento de E/S en redes de area local.

8. Referencias Bibliograficas

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). John Wiley & Sons.

Tanenbaum, A. S., & Bos, H. (2015). Modern Operating Systems (4th ed.). Pearson.

Love, R. (2010). Linux Kernel Development (3rd ed.). Addison-Wesley Professional.

Russinovich, M., Solomon, D. A., & Ionescu, A. (2017). Windows Internals, Part 1: System architecture, processes, threads, memory management, and more (7th ed.). Microsoft Press.

Samba Team. (2024). Samba: Opening Windows to a Wider World (Documentation & smb.conf manual). https://www.samba.org